# 指示登録タブ UI仕様書

## 概要

### 目的

プロジェクトに登録された指示の一覧表示と管理を行うタブコンポーネント。指示の新規作成・編集・削除、および一括送信機能を提供する。

### 使用場面

- プロジェクト管理画面の「指示登録」タブ
- 実験管理者がワーカーに対する指示を管理する際に使用
- タスク登録後、指示の確認・追加・送信を行う

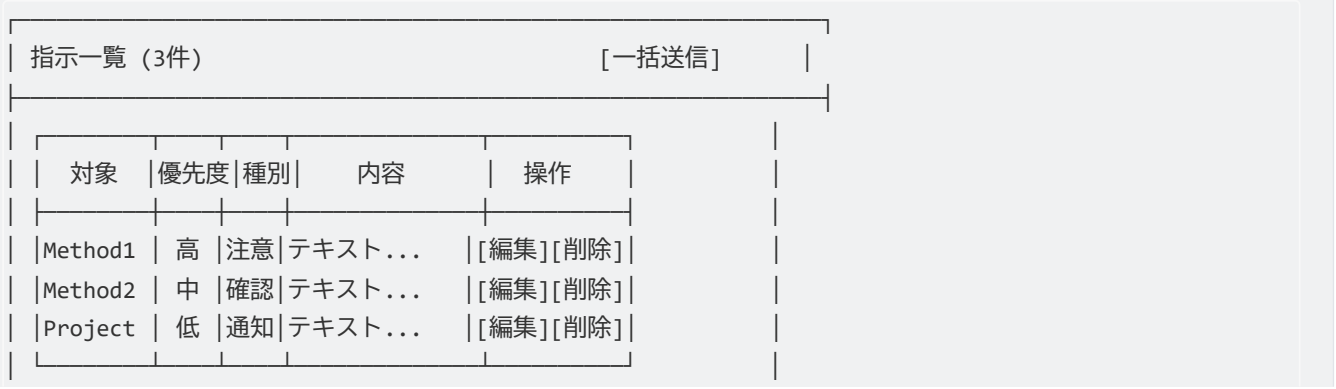
### アクセス権限

- **Experiment Admin（実験管理者）**: 指示の閲覧・作成・編集・削除・送信
- **その他のロール**: アクセス不可

---

## 画面構成

### レイアウト構造



```
└──────────────────────────────────────────────────────────┘

【送信済み指示の表示】
┌──────────────────────────────────────────────────────────┐
│ │Method1 │ 高 │注意│テキスト...    │ [詳細] │ ← 送信済 │
│ └────────┴────┴────┴──────────────┴────────┘          │
└──────────────────────────────────────────────────────────┘

【モーダル】
┌──────────────────────────────────────┐
│ 指示編集                          [×]│
├──────────────────────────────────────┤
│ 対象: [メソッド選択▼]                 │
│ 種別: [選択▼]                        │
│ 優先度: [選択▼]                      │
│ 内容: [テキストエリア]                │
│                                      │
│           [キャンセル] [保存]        │
└──────────────────────────────────────┘
```

# UI要素

## 1. ツールバー

- **指示件数表示**: 「指示一覧 ({count}件)」（左揃え、グレーテキスト）
- **一括送信ボタン**: 全指示をワーカーに送信
  - 有効条件: 指示が1件以上、チェックアウトされていない、送信中でない
  - 色: 緑色（`token.colorSuccess`）
  - アイコン: `SendOutlined`

## 2. 指示一覧テーブル

| カラム | 説明 | 幅 | 表示内容 |
|---|---|---|---|
| 対象 | 指示の対象（プロジェクト/メソッド/Transition/タスク） | 200px | テキスト |
| 優先度 | 指示の優先度（高/中/通常/低） | 100px | タグ（色付き） |
| 種別 | 指示の種別 | 120px | テキスト |
| 内容 | 指示の詳細内容 | - | テキスト（ellipsis、Tooltip） |

| | | | |
|---|---|---|---|
| 操作 | 編集・削除・詳細ボタン | 150px | ボタングループ |

### 3. 操作ボタンの表示分岐

**未送信指示（`isSent = false`）:**

- **編集ボタン**: 指示を編集
- **削除ボタン**: 指示を削除（danger スタイル）

**送信済み指示（`isSent = true`）:**

- **詳細ボタン**: 指示を閲覧（編集・削除は不可）

### 4. 空状態

- 指示が0件の場合: `Empty.PRESENTED_IMAGE_SIMPLE`
- メッセージ: `instructionRegistration.emptyState`

---

# 機能仕様

## Props定義

```
interface InstructionRegistrationTabProps {
  projectId: string;                    // プロジェクトID
  projectName: string;                  // プロジェクト名
  isActive: boolean;                    // タブがアクティブか
  isCheckoutLocked?: boolean;           // チェックアウト状態
  onInstructionsSubmit?: (instructions: InstructionData[]) => void; // 送信完了時のコールバック
}
```

## State管理

### 主要なState

```
// 指示一覧
const [instructions, setInstructions] = useState<InstructionData[]>([]);

// 登録済みタスク（InstructionEditDialogに渡す）
const [registeredTasks, setRegisteredTasks] = useState<TaskDetail[]>([]);

// ダイアログ状態
const [editDialogOpen, setEditDialogOpen] = useState<boolean>(false);
const [editMode, setEditMode] = useState<'create' | 'edit'>('create');
const [editingInstruction, setEditingInstruction] = useState<InstructionData | undefined>
(undefined);

// 送信状態
const [isSending, setIsSending] = useState<boolean>(false);
```

## Context連携

```
const { t } = useLanguage();                              // 多言語対応
const { maimlRoot, isLoading: isMaimlLoading } = useProjectMaiml(); // MAIMLデータ取得
```

## useMemoによるデータ取得

### 1. メソッド一覧（MAIMLコンテキストから）

```
const availableMethods = useMemo((): MethodInfo[] => {
  if (!maimlRoot?.protocol?.methods) return [];

  return maimlRoot.protocol.methods.map((method) => ({
    id: method.id,
    name: method.name || method.id,
    displayName: method.name || method.id,
    description: method.description,
  }));
}, [maimlRoot]);
```

### 2. トランジション一覧（MAIMLコンテキストから）

```
const availableTransitions = useMemo((): TransitionInfo[] => {
  if (!maimlRoot?.protocol?.methods) return [];

  // 全メソッドのトランジションを集約
  const allTransitions: TransitionInfo[] = [];
  maimlRoot.protocol.methods.forEach((method) => {
    if (method.pnml?.transitions) {
      method.pnml.transitions.forEach((transition) => {
        allTransitions.push({
          id: transition.id,
          displayName: transition.name || transition.id,
          category: 'other',
          description: transition.description,
          isExperimentRelevant: true,
        });
      });
    }
  });

  return allTransitions;
}, [maimlRoot]);
```

**注**: InstructionEditDialogではメソッド選択時に動的にフィルタリングされます。

# 主要機能

## 1. タスク登録情報の取得（タブアクティブ時）

**実行タイミング**: タブがアクティブになった時（`isActive = true`）

**処理フロー**:

```
useEffect(() => {
  if (!projectName || !isActive) return;

  // タスク登録データをAPI取得
  const data = await getTaskRegistration(projectName);

  // タスク情報をUI用の形式に変換
  if (data.tasks && data.tasks.length > 0) {
    const uiTasks: TaskDetail[] = data.tasks.map((task, index) => ({
      id: index + 1,
      methodId: task.methodId,
```

```
      experimentCondition: Number(task.experimentCondition),
      trialCount: task.trialCount,
      experimentOrder: task.experimentOrder,
      taskNumberOptions: Array.from({ length: task.trialCount }, (_, i) => i + 1),
      workerLinkedFlg: task.workerLinkedFlg || false,
    }));

    setRegisteredTasks(uiTasks);
  } else {
    setRegisteredTasks([]);
  }
}, [projectName, isActive]);
```

**API**: `GET /api/v1/experiment-admin/project/:projectName/task-registration`

## 2. 指示一覧の取得（タブアクティブ時）

**実行タイミング**: タブがアクティブになった時（`isActive = true`）

**処理フロー**:

```
useEffect(() => {
  if (!projectName || !isActive) return;

  const instructionList = await getInstructions(projectName);
  setInstructions(instructionList);
}, [projectName, isActive]);
```

**API**: `GET /api/v1/experiment-admin/project/:projectName/instructions`

**レスポンス**:

```
Array<{
  id: string | number;
  methodId: string;
  transitionId?: string | null;
  taskNumber?: number;
  priority: number;
  type: number;
  content: string;
```

```
  isSent: boolean;
}>
```

## 3. 新規指示追加ダイアログを開く

```
const handleOpenCreateDialog = useCallback(() => {
  setEditMode('create');
  setEditingInstruction(undefined);
  setEditDialogOpen(true);
}, []);
```

**注**: 現在の実装では、新規追加ボタンは表示されていません。新規追加はタスク登録タブから行います。

## 4. 指示編集ダイアログを開く

```
const handleOpenEditDialog = useCallback((instruction: InstructionData) => {
  setEditMode('edit');
  setEditingInstruction(instruction);
  setEditDialogOpen(true);
}, []);
```

**送信済み指示の場合**: 詳細表示モードで開く（編集・削除は不可）

## 5. 指示の保存

**実行フロー**:

```
const handleSaveInstruction = useCallback(async (instruction: InstructionData, keepDialogOpen:
boolean = false) => {
  try {
    if (editMode === 'create') {
      // 新規作成：API呼び出し
      const result = await saveInstruction(projectName, {
        methodId: instruction.methodId,
        transitionId: instruction.transitionId || null,
        taskNumber: instruction.taskNumber,
        priority: instruction.priority,
        type: instruction.type,
        content: instruction.content,
```

```
      });

      // APIから返されたIDで新しい指示を追加
      const newInstruction: InstructionData = {
        ...instruction,
        id: result.instructionId,
        isSent: false,
      };

      setInstructions((prev) => [...prev, newInstruction]);
      message.success(t('instructionRegistration.messages.added'));
    } else {
      // 編集：API呼び出し
      await updateInstruction(projectName, String(instruction.id), {
        methodId: instruction.methodId,
        transitionId: instruction.transitionId || null,
        taskNumber: instruction.taskNumber,
        priority: instruction.priority,
        type: instruction.type,
        content: instruction.content,
      });

      // ローカル状態を更新
      setInstructions((prev) =>
        prev.map((inst) => (inst.id === instruction.id ? instruction : inst))
      );
      message.success(t('instructionRegistration.messages.updated'));
    }

    // 連続指示の場合はダイアログを閉じない
    if (!keepDialogOpen) {
      handleCloseDialog();
    } else {
      // 連続指示モード：新規作成モードに切り替え
      setEditMode('create');
      setEditingInstruction(undefined);
    }
  } catch (error) {
    const errorMessage =
      error instanceof Error ? error.message :
t('instructionRegistration.messages.saveFailed');
    message.error(errorMessage);
  }
}, [editMode, projectName, handleCloseDialog, t]);
```

**API（新規作成）**：`POST /api/v1/experiment-admin/project/:projectName/instruction`

**リクエスト**:

```
{
  methodId: string;
  transitionId: string | null;
  taskNumber?: number;
  priority: number;
  type: number;
  content: string;
}
```

**API（編集）**: `PUT /api/v1/experiment-admin/project/:projectName/instruction/:id`

**連続指示モード**:

- `keepDialogOpen = true` の場合、保存後もダイアログを閉じない
- 新規作成モードに切り替えて次の指示を連続入力可能

## 6. 指示の削除

```
const handleDeleteInstruction = useCallback(async (id: string | number) => {
  try {
    // API呼び出し
    const result = await deleteInstruction(projectName, String(id));

    if (result.deleted) {
      // ローカル状態を更新
      setInstructions((prev) => prev.filter((inst) => inst.id !== id));
      message.success(t('instructionRegistration.messages.deleted'));
    } else {
      message.error(t('instructionRegistration.messages.deleteFailed'));
    }
  } catch (error) {
    const errorMessage =
      error instanceof Error ? error.message :
t('instructionRegistration.messages.deleteError');
    message.error(errorMessage);
  }
}, [projectName]);
```

**API**: `DELETE /api/v1/experiment-admin/project/:projectName/instruction/:id`

**レスポンス**:

```
{
  deleted: boolean;
}
```

## 7. 指示の一括送信

**実行フロー**:

```
const handleSubmitInstructions = useCallback(async () => {
  // バリデーション: 指示が0件の場合
  if (instructions.length === 0) {
    message.warning(t('instructionRegistration.messages.noInstructions'));
    return;
  }

  try {
    setIsSending(true);

    // 指示を一括送信
    const result = await sendInstructions(projectName, {});

    // 送信済フラグを立てる
    const sentInstructions = instructions.map(inst => ({
      ...inst,
      isSent: true,
    }));

    // 親コンポーネントに通知
    onInstructionsSubmit?.(sentInstructions);

    message.success(t('instructionRegistration.messages.sent').replace('{count}',
result.sentInstructionsCount.toString()));

    // 送信後、送信済フラグを立てて表示を継続
    setInstructions(sentInstructions);
  } catch (error) {
    const errorMessage =
      error instanceof Error ? error.message :
t('instructionRegistration.messages.sendFailed');
```

```
    message.error(errorMessage);
  } finally {
    setIsSending(false);
  }
}, [instructions, projectName, onInstructionsSubmit, t]);
```

**API**: `POST /api/v1/experiment-admin/project/:projectName/instructions/send`

**リクエスト**:

```
{} // 空オブジェクト
```

**レスポンス**:

```
{
  sentInstructionsCount: number;
}
```

**送信後の処理**:

- 全指示に `isSent = true` フラグを立てる
- `onInstructionsSubmit` コールバック実行
- 成功メッセージ表示（送信件数付き）
- 指示一覧は送信後も表示を継続（詳細ボタンのみ）

## 8. 表示用ヘルパー関数

**優先度のタグカラー**:

```
const getPriorityColor = (priority: number): string => {
  switch (priority) {
    case 3: // 高
      return 'red';
    case 2: // 中
      return 'orange';
    case 1: // 通常
      return 'blue';
```

```
    case 0: // 低
      return 'default';
    default:
      return 'default';
  }
};
```

**優先度のラベル**:

```
const getPriorityLabel = useCallback((priority: number): string => {
  return t(`instructionRegistration.priority.${priority}`) || String(priority);
}, [t]);
```

**種別のラベル**:

```
const getTypeLabel = useCallback((type: number): string => {
  return t(`instructionRegistration.type.${type}`) || String(type);
}, [t]);
```

**対象のラベル**:

```
const getTargetLabel = useCallback((instruction: InstructionData): string => {
  const method = availableMethods.find(m => m.id === instruction.methodId);

  if (instruction.transitionId) {
    const transition = availableTransitions.find(t => t.id === instruction.transitionId);
    return `${method?.displayName || t('instructionRegistration.targetLabels.method')} >
${transition?.displayName || t('instructionRegistration.targetLabels.transition')}`;
  } else if (instruction.taskNumber !== undefined) {
    return `${method?.displayName || t('instructionRegistration.targetLabels.method')} >
${t('instructionRegistration.targetLabels.task')} ${instruction.taskNumber}`;
  } else if (instruction.methodId) {
    return method?.displayName || t('instructionRegistration.targetLabels.method');
  } else {
    return t('instructionRegistration.targetLabels.wholeProject');
  }
}, [availableMethods, availableTransitions, t]);
```

**対象ラベルの階層表示**:

- プロジェクト全体: 「プロジェクト全体」
- メソッド: 「{メソッド名}」
- メソッド > タスク: 「{メソッド名} > タスク {番号}」
- メソッド > Transition: 「{メソッド名} > {Transition名}」

---

# データ型定義

## InstructionData

```
interface InstructionData {
  id: string | number;                // 指示ID
  methodId: string;                   // メソッドID（空文字の場合はプロジェクト指示）
  transitionId?: string | null;       // TransitionID
  taskNumber?: number;                // タスク番号
  priority: number;                   // 優先度（0: 低, 1: 通常, 2: 中, 3: 高）
  type: number;                       // 種別
  content: string;                    // 指示内容
  targetType?: string;                // 対象種別（表示用）
  isSent?: boolean;                   // 送信済みフラグ
}
```

## TaskDetail

```
interface TaskDetail {
  id: number;                         // UI内部ID
  methodId: string;                   // メソッドID
  experimentCondition: number;        // 実験条件
  trialCount: number;                 // 試行回数
  experimentOrder: number;            // 作業順序
  taskNumberOptions: number[];        // タスク番号のオプション
  workerLinkedFlg: boolean;           // ワーカー連携済みフラグ
}
```

## MethodInfo

```
interface MethodInfo {
  id: string;                          // メソッドID
  name: string;                        // メソッド名
  displayName: string;                 // 表示名
  description?: string;                // 説明
}
```

### TransitionInfo

```
interface TransitionInfo {
  id: string;                          // TransitionID
  displayName: string;                 // 表示名
  category: string;                    // カテゴリ
  description?: string;                // 説明
  isExperimentRelevant: boolean;       // 実験関連フラグ
}
```

---

## 子コンポーネント

### InstructionEditDialog

**用途**: 指示の新規作成・編集・詳細表示ダイアログ

**Props**:

```
{
  open: boolean;                       // ダイアログ表示状態
  mode: 'create' | 'edit';             // モード（作成/編集）
  instruction?: InstructionData;       // 編集対象の指示
  tasks: TaskDetail[];                 // タスク一覧
  availableMethods: MethodInfo[];      // メソッド一覧
  availableTransitions: TransitionInfo[]; // Transition一覧
  onClose: () => void;                 // 閉じるハンドラー
  onSave: (instruction: InstructionData, keepDialogOpen?: boolean) => void; // 保存ハンドラー
}
```

**使用箇所**:

- 新規作成: `mode='create'`, `instruction=undefined`
- 編集: `mode='edit'`, `instruction={editingInstruction}`
- 詳細表示: `mode='edit'`, `instruction={editingInstruction}`, `isSent=true`（編集・削除不可）

---

# エラーハンドリング

## エラーメッセージ

| エラー条件 | メッセージキー | 表示メッセージ |
|---|---|---|
| タスク取得失敗 | `instructionRegistration.messages.taskFetchFailed` | 「タスク情報の取得に失敗しました」 |
| 指示一覧取得失敗 | `instructionRegistration.messages.fetchFailed` | 「指示一覧の取得に失敗しました」 |
| 指示が0件で送信 | `instructionRegistration.messages.noInstructions` | 「送信する指示がありません」 |
| 指示追加成功 | `instructionRegistration.messages.added` | 「指示を追加しました」 |
| 指示更新成功 | `instructionRegistration.messages.updated` | 「指示を更新しました」 |
| 指示保存失敗 | `instructionRegistration.messages.saveFailed` | 「指示の保存に失敗しました」 |
| 指示削除成功 | `instructionRegistration.messages.deleted` | 「指示を削除しました」 |
| 指示削除失敗 | `instructionRegistration.messages.deleteFailed` | 「指示の削除に失敗しました」 |
| 指示削除エラー | `instructionRegistration.messages.deleteError` | 「削除処理でエラーが発生しました」 |
| 指示送信成功 | `instructionRegistration.messages.sent` | 「{count}件の指示を送信しました」 |
| 指示送信失敗 | `instructionRegistration.messages.sendFailed` | 「指示の送信に失敗しました」 |

## ローディング状態

**MAIML読み込み中**:

- テーブルに `loading={isMaimlLoading}` プロップを設定
- ローディングスピナー表示

**送信中**:

- 一括送信ボタンに `loading={isSending}` プロップを設定
- 編集・削除ボタンを非活性化

---

# 特記事項

## 送信済み指示の扱い

**送信済み指示（ `isSent = true` ）**:

- 編集・削除ボタンを非表示
- 詳細ボタンのみ表示
- 詳細ボタンクリック時は閲覧モードでダイアログを開く
- 送信後も指示一覧に表示を継続

## 指示の対象階層

**対象の優先順位**:

1. **Transition指定あり**: 「{メソッド名} > {Transition名}」
2. **タスク番号指定あり**: 「{メソッド名} > タスク {番号}」
3. **メソッド指定のみ**: 「{メソッド名}」
4. **プロジェクト全体**: 「プロジェクト全体」

## MAIMLコンテキストとの連携

**メソッド一覧**:

- `maimlRoot.protocol.methods` から取得

- `useMemo` でメモ化
- InstructionEditDialogに渡す

**トランジション一覧**:

- 全メソッドの `pnml.transitions` を集約
- `useMemo` でメモ化
- InstructionEditDialogに渡す
- ダイアログ内でメソッド選択時に動的にフィルタリング

## 連続指示モード

**keepDialogOpen = true の場合**:

- 保存後もダイアログを閉じない
- 新規作成モードに切り替え
- 続けて次の指示を入力可能
- InstructionEditDialog側で実装

## 一括送信の仕様

**送信対象**:

- 現在表示されている全指示
- 送信済み指示も含む（バックエンドで重複送信を防止）

**送信後の状態**:

- 全指示に `isSent = true` フラグを立てる
- 指示一覧は表示を継続
- 編集・削除ボタンを詳細ボタンに切り替え

## タスク登録情報の取得タイミング

**実行タイミング**:

- タブがアクティブになった時（ `isActive = true` ）

- InstructionEditDialogでタスク番号の選択肢を提供するために必要

**取得内容**:

- タスクのメソッドID
- 試行回数（タスク番号の上限）
- ワーカー連携済みフラグ

---

# API連携

## 1. タスク登録情報取得

**エンドポイント**: `GET /api/v1/experiment-admin/project/:projectName/task-registration`

**レスポンス**:

```
{
  isMaimlRegistered: boolean;
  tasks: Array<{
    methodId: string;
    experimentCondition: string;
    trialCount: number;
    experimentOrder: number;
    workerLinkedFlg: boolean;
  }>;
}
```

## 2. 指示一覧取得

**エンドポイント**: `GET /api/v1/experiment-admin/project/:projectName/instructions`

**レスポンス**:

```
Array<{
  id: string | number;
```

```
  methodId: string;
  transitionId?: string | null;
  taskNumber?: number;
  priority: number;
  type: number;
  content: string;
  isSent: boolean;
}>
```

## 3. 指示新規作成

**エンドポイント**: `POST /api/v1/experiment-admin/project/:projectName/instruction`

**リクエスト**:

```
{
  methodId: string;
  transitionId: string | null;
  taskNumber?: number;
  priority: number;
  type: number;
  content: string;
}
```

**レスポンス**:

```
{
  instructionId: string | number;
}
```

## 4. 指示編集

**エンドポイント**: `PUT /api/v1/experiment-admin/project/:projectName/instruction/:id`

**リクエスト**:

```
{
  methodId: string;
  transitionId: string | null;
  taskNumber?: number;
  priority: number;
  type: number;
  content: string;
}
```

## 5. 指示削除

**エンドポイント**: `DELETE /api/v1/experiment-admin/project/:projectName/instruction/:id`

**レスポンス**:

```
{
  deleted: boolean;
}
```

## 6. 指示一括送信

**エンドポイント**: `POST /api/v1/experiment-admin/project/:projectName/instructions/send`

**リクエスト**:

```
{} // 空オブジェクト
```

**レスポンス**:

```
{
  sentInstructionsCount: number;
}
```

---

# 実装ファイル情報

## パス

```
frontend/src/components/project/InstructionRegistrationTab.tsx
```

## 主要な依存パッケージ

- `antd` : UI コンポーネント（Card, Button, Table, Space, Tag, Typography, message, theme, Tooltip, Empty）
- `@ant-design/icons` : アイコン（PlusOutlined, EditOutlined, DeleteOutlined, SendOutlined, EyeOutlined）
- `react` : React本体（useState, useEffect, useCallback, useMemo）

## Context

- `LanguageContext` : 多言語対応
- `ProjectMaimlContext` : MAIMLデータ取得

## API Service

- `projectApiService` : 指示関連のAPI呼び出し

---

# 変更履歴

| バージョン | 日付 | 変更内容 | 担当者 |
|---|---|---|---|
| 1.0 | 2026-03-11 | 実装ベースで仕様書を新規作成（実装ファイル: InstructionRegistrationTab.tsx） | Cline |

---

# 関連ドキュメント

- プロジェクト管理画面
- タスク登録タブ
- 指示編集ダイアログ
- タスク結果一覧テーブル